

Estructura de Datos - Módulo 3

Introducción a estructuras no lineales orientadas a C++

Aplicación al incidente crítico PAD-Bienestar

Idea clave: No toda la información se organiza en fila. En C++, los árboles permiten representar jerarquías y los grafos permiten representar redes, rutas, dependencias y relaciones entre varios elementos.

1. Enfoque general del Módulo 3 orientado a C++

El Módulo 3 amplía la visión de las estructuras de datos: ya no se trabaja únicamente con listas, pilas o colas, donde cada elemento se relaciona con el siguiente. Ahora se estudian estructuras no lineales, en las que un dato puede estar conectado con varios datos al mismo tiempo. Esta idea es importante porque muchos sistemas reales no se comportan como una fila, sino como jerarquías, redes o dependencias.

Desde el enfoque de Deitel & Deitel en C++ Cómo programar, 9.ª edición, una buena solución no solo depende de escribir instrucciones correctas, sino de elegir una forma adecuada de organizar los datos y las operaciones. En C++, las estructuras no lineales pueden modelarse mediante struct, apuntadores, funciones recursivas, arreglos, vectores y matrices.

Relación con el PDF: El contenido nuclear del Módulo 3 aborda árboles, árboles binarios, recorridos, árboles binarios de búsqueda, grafos, grafos dirigidos/no dirigidos, matriz de adyacencia, lista de adyacencia y aplicaciones reales. En este archivo se adapta todo a C++ y al caso PAD-Bienestar.

Necesidad del sistema	Estructura en C++	Razón principal
Representar módulos y sedes	Árbol	Organiza información por niveles.
Representar nodos con dos ramas	Árbol binario	Cada nodo tiene hijo izquierdo y derecho.
Buscar IDs ordenados	Árbol binario de búsqueda	Reduce comparaciones durante la búsqueda.
Procesar árboles	Recorridos	Permiten visitar todos los nodos.
Modelar red de equipos	Grafo	Representa conexiones entre dispositivos.
Representar dependencias	Grafo dirigido	La relación tiene sentido u orden.
Verificar conexiones rápidas	Matriz de adyacencia	Consulta directa entre dos nodos.
Ahorrar memoria en redes dispersas	Lista de adyacencia	Guarda solo conexiones existentes.

2. Estructuras no lineales

Teoría orientada a C++

Una estructura no lineal organiza datos de forma más compleja que una secuencia. En una lista, cada nodo tiene un siguiente; en una pila se trabaja con el tope; en una cola se trabaja con frente y final. En cambio, en un árbol un nodo puede tener varios hijos, y en un grafo un nodo puede conectarse con muchos otros nodos.

Aplicación PAD-Bienestar: Aunque el sistema base puede usar listas y colas, algunas relaciones del caso pueden modelarse mejor con estructuras no lineales: módulos del sistema, sedes, equipos, red de conexión, rutas de atención o dependencias entre procesos.

Ejemplo base en C++

```
// Una estructura no lineal permite que un elemento
// se relacione con varios elementos.
struct NodoGeneral {
    string nombre;
    vector<NodoGeneral*> hijos;
};
```

Imagen de apoyo visual

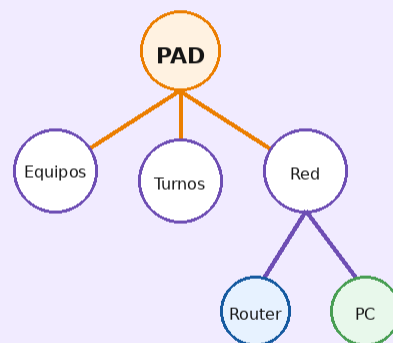
c++ Estructuras no lineales

Estructura lineal



Un dato va después de otro: listas, pilas o colas.

Estructura no lineal



Un dato puede relacionarse con varios: árboles, grafos, jerarquías y redes.

Imagen 1. Diferencia entre una estructura lineal y una estructura no lineal.

3. Árboles en C++

Teoría orientada a C++

Un árbol es una estructura no lineal que organiza datos de forma jerárquica. Tiene un nodo raíz, nodos hijos, nodos padre y nodos hoja. La raíz es el punto principal desde donde se desprenden los demás elementos. En C++, un árbol general puede representarse con un struct que tenga un dato y una colección de apuntadores a sus hijos.

Aplicación PAD-Bienestar: En PAD-Bienestar, un árbol puede representar la jerarquía del sistema: programa PAD, sede, equipos, usuarios, sesiones o servicios disponibles. Esta estructura ayuda cuando la información tiene niveles claros.

Ejemplo base en C++

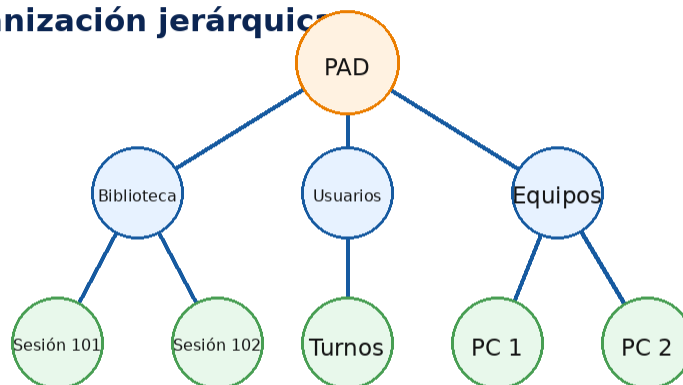
```
#include <iostream>
#include <vector>
using namespace std;

struct NodoArbol {
    string nombre;
    vector<NodoArbol*> hijos;
};

NodoArbol* crearNodo(string nombre) {
    NodoArbol* nuevo = new NodoArbol;
    nuevo->nombre = nombre;
    return nuevo;
}
```

c++ Árboles en C++

Árbol: organización jerárquica



Raíz: PAD. Hijos: Biblioteca, Usuarios y Equipos. Hojas: sesiones, turnos y equipos específicos. La estructura no tiene ciclos y cada nodo tiene un solo padre.

Imagen 2. Árbol aplicado a una jerarquía del PAD-Bienestar.

4. Árbol binario en C++

Teoría orientada a C++

Un árbol binario es un tipo de árbol donde cada nodo puede tener como máximo dos hijos: hijo izquierdo e hijo derecho. Si un hijo no existe, el apuntador queda en nullptr. Esta estructura es importante porque sirve como base para árboles binarios de búsqueda y otros algoritmos.

Aplicación PAD-Bienestar: En PAD-Bienestar, puede usarse de manera didáctica para organizar identificadores o prioridades en dos caminos de decisión: izquierda y derecha. Por ejemplo, decisiones de búsqueda por número de equipo o por identificador de sesión.

Ejemplo base en C++

```
struct NodoBinario {
    int dato;
    NodoBinario* izquierdo;
    NodoBinario* derecho;
};

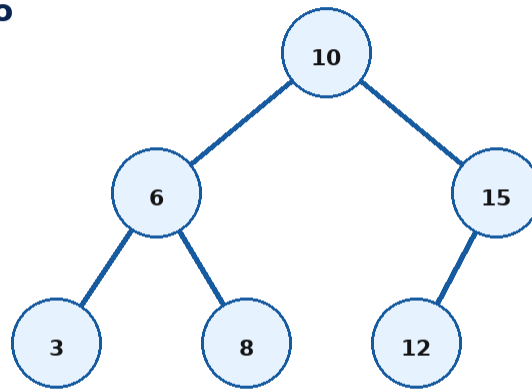
NodoBinario* crearNodo(int dato) {
    NodoBinario* nuevo = new NodoBinario;
    nuevo->dato = dato;
    nuevo->izquierdo = nullptr;
    nuevo->derecho = nullptr;
    return nuevo;
}
```

```
}
```

Imagen de apoyo visual

c++ **Árbol binario en C++**

Árbol binario



Cada nodo puede tener como máximo dos hijos: izquierdo y derecho.

En C++ se representa con dos apuntadores: izquierdo y derecho.

Imagen 3. Árbol binario: cada nodo tiene como máximo dos hijos.

5. Recorridos básicos de árboles

Teoría orientada a C++

Recorrer un árbol significa visitar todos sus nodos una vez. En árboles binarios existen tres recorridos básicos: preorden, inorden y postorden. Preorden procesa primero la raíz; inorden procesa primero el subárbol izquierdo, luego la raíz y después el derecho; postorden procesa primero los hijos y al final la raíz.

Aplicación PAD-Bienestar: En PAD-Bienestar, los recorridos pueden servir para consultar una jerarquía completa de módulos, generar reportes ordenados o eliminar nodos de memoria de manera segura procesando primero los hijos.

Ejemplo base en C++

```
void preorden(NodoBinario* raiz) {
    if (raiz == nullptr) return;
    cout << raiz->dato << " ";
    preorden(raiz->izquierdo);
    preorden(raiz->derecho);
}

void inorden(NodoBinario* raiz) {
    if (raiz == nullptr) return;
```

```

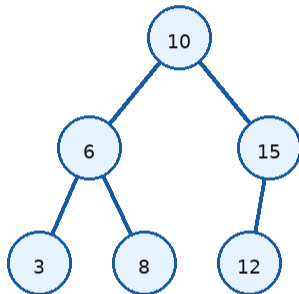
inorden(raiz->izquierdo);
cout << raiz->dato << " ";
inorden(raiz->derecho);
}

```

Imagen de apoyo visual

c++ Recorridos de árboles

Recorridos básicos en árboles



Preorden

raíz -> izquierdo -> derecho
10, 6, 3, 8, 15, 12

Inorden

izquierdo -> raíz -> derecho
3, 6, 8, 10, 12, 15

Postorden

izquierdo -> derecho -> raíz
3, 8, 6, 12, 15, 10

En C++, estos recorridos suelen implementarse con funciones recursivas: la función se llama a sí misma para visitar el subárbol izquierdo y derecho.

Imagen 4. Recorridos preorden, inorden y postorden en un árbol binario.

6. Árbol binario de búsqueda en C++

Teoría orientada a C++

Un árbol binario de búsqueda, también llamado ABB, es un árbol binario con una regla de orden: los valores menores se colocan a la izquierda y los valores mayores a la derecha. Esta propiedad permite realizar búsquedas más eficientes que en una lista lineal, especialmente cuando el árbol se mantiene equilibrado.

Aplicación PAD-Bienestar: En PAD-Bienestar, un ABB puede usarse para buscar identificadores de sesiones, usuarios o equipos. Por ejemplo, si se desea localizar rápidamente el ID de una sesión activa, el árbol permite decidir si se avanza a la izquierda o a la derecha en cada comparación.

Ejemplo base en C++

```

NodoBinario* insertar(NodoBinario* raiz, int dato) {
    if (raiz == nullptr) {
        return crearNodo(dato);
    }
    if (dato < raiz->dato) {
        raiz->izquierdo = insertar(raiz->izquierdo, dato);
    }
}

```

```

    } else if (dato > raiz->dato) {
        raiz->derecho = insertar(raiz->derecho, dato);
    }
    return raiz;
}

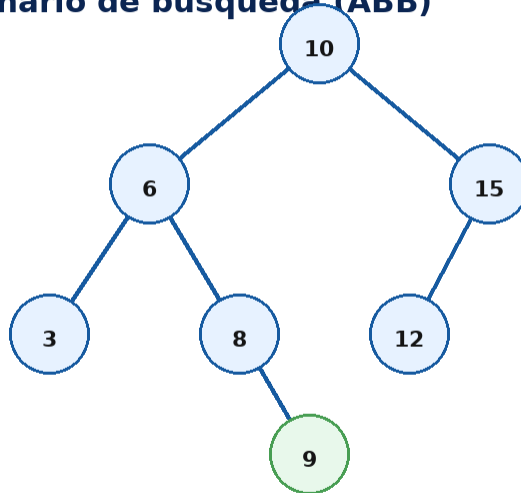
bool buscar(NodoBinario* raiz, int dato) {
    if (raiz == nullptr) return false;
    if (raiz->dato == dato) return true;
    if (dato < raiz->dato) return buscar(raiz->izquierdo, dato);
    return buscar(raiz->derecho, dato);
}

```

Imagen de apoyo visual

c++ Árbol binario de búsqueda

Árbol binario de búsqueda (ABB)



Regla del ABB

Menores a la izquierda.
Mayores a la derecha. La
búsqueda compara y
descarta ramas.

Insertar 9

9 < 10: izquierda
9 > 6: derecha
9 > 8: derecha
Espacio vacío: insertar.

Imagen 5. Árbol binario de búsqueda: regla de menores a la izquierda y mayores a la derecha.

7. Grafos en C++

Teoría orientada a C++

Un grafo es una estructura de datos formada por nodos o vértices y conexiones llamadas aristas. A diferencia de un árbol, un grafo no necesariamente tiene una raíz ni una jerarquía única. Un nodo puede conectarse con muchos otros nodos, lo que permite modelar redes, rutas, relaciones y sistemas interconectados.

Aplicación PAD-Bienestar: En PAD-Bienestar, un grafo puede representar la red de equipos, routers, servidores e impresoras. Cada dispositivo es un nodo y cada conexión física o inalámbrica es una arista.

Ejemplo base en C++

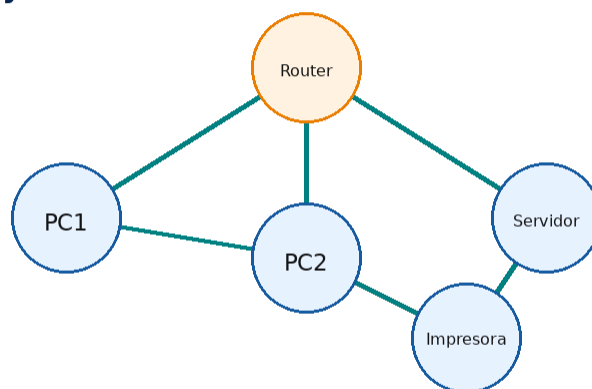
```
struct Arista {
    int origen;
    int destino;
};

// Ejemplo: 0=Router, 1=PC1, 2=PC2, 3=Servidor
vector<Arista> conexiones = {
    {0, 1}, {0, 2}, {0, 3}, {2, 3}
};
```

Imagen de apoyo visual

c++ Grafos en C++

Grafo: nodos y aristas



Nodo / vértice

Elemento del sistema: equipo, usuario, router o servidor.

Arista

Conexión o relación entre nodos. Puede tener dirección o peso.

Imagen 6. Grafo aplicado a conexiones de red del PAD-Bienestar.

8. Grafos dirigidos y no dirigidos

Teoría orientada a C++

Un grafo no dirigido representa relaciones en ambos sentidos. Si A se conecta con B, entonces B también se conecta con A. En cambio, un grafo dirigido representa relaciones con sentido: A puede apuntar a B sin que B apunte a A. Esto es útil para modelar dependencias, rutas de una sola dirección o procesos con orden.

Aplicación PAD-Bienestar: En PAD-Bienestar, una red local puede modelarse como grafo no dirigido si la comunicación es bidireccional. En cambio, las dependencias del proceso de atención pueden modelarse como grafo dirigido: registrar usuario, asignar turno, abrir sesión y cerrar sesión.

Ejemplo base en C++

```
// Grafo dirigido: una tarea apunta a la siguiente
vector<pair<string, string>> dependencias = {
    {"Registrar usuario", "Asignar turno"},
    {"Asignar turno", "Abrir sesion"},
    {"Abrir sesion", "Cerrar sesion"}
};
```

Imagen de apoyo visual

c++ Tipos de grafos

Grafo dirigido y no dirigido

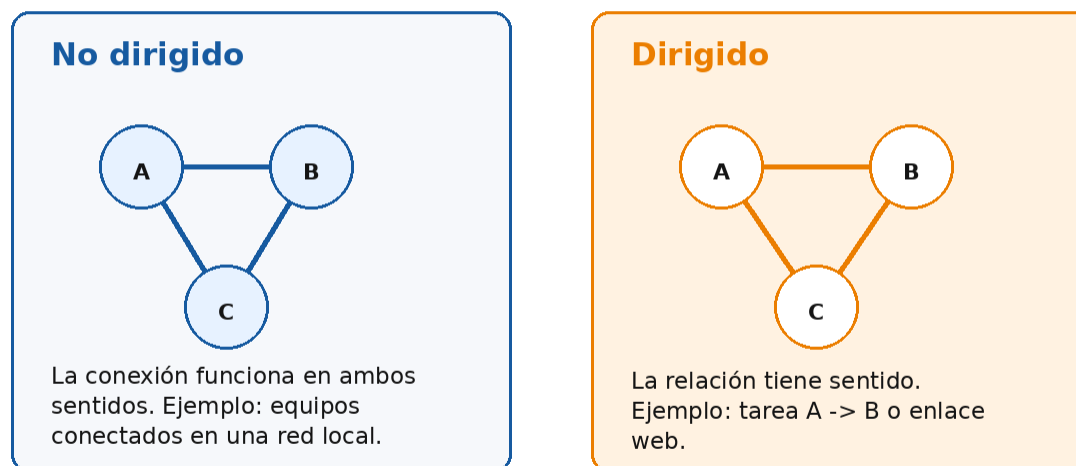


Imagen 7. Diferencia visual entre grafo no dirigido y grafo dirigido.

9. Matriz de adyacencia en C++

Teoría orientada a C++

La matriz de adyacencia representa un grafo mediante una tabla de n por n . Cada fila y columna corresponde a un nodo. Si existe conexión entre dos nodos, se coloca 1; si no existe conexión, se coloca 0. En grafos con peso, se puede colocar el valor del peso en lugar de 1.

Aplicación PAD-Bienestar: En PAD-Bienestar, una matriz de adyacencia puede representar si existe conexión entre equipos, router, servidor o impresora. Es útil cuando se quiere verificar rápidamente si dos elementos están conectados.

Ejemplo base en C++

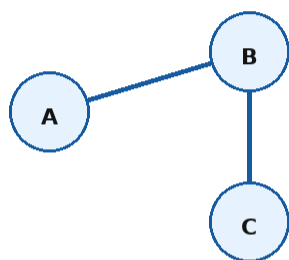
```
const int N = 4;
int red[N][N] = {
    // R  PC1 PC2 Servidor
    {0, 1, 1, 1}, // Router
    {1, 0, 0, 0}, // PC1
    {1, 0, 0, 1}, // PC2
    {1, 0, 1, 0}  // Servidor
};

if (red[0][2] == 1) {
    cout << "Router conectado con PC2";
}
```

Imagen de apoyo visual

c++ Matriz de adyacencia

Matriz de adyacencia



Grafo: A-B-C

	A	B	C
A	0	1	0
B	1	0	1
C	0	1	0

En C++, se implementa como arreglo bidimensional o `vector<vector<int>>`. Es rápida para verificar si dos nodos están conectados.

Imagen 8. Matriz de adyacencia para representar conexiones entre nodos.

10. Lista de adyacencia en C++

Teoría orientada a C++

La lista de adyacencia representa un grafo guardando, para cada nodo, la lista de nodos vecinos con los que está conectado. A diferencia de la matriz, no guarda espacios para conexiones inexistentes, por lo que suele ahorrar memoria en grafos grandes con pocas conexiones.

Aplicación PAD-Bienestar: En PAD-Bienestar, una lista de adyacencia puede representar solo las conexiones reales entre los dispositivos de red, lo cual resulta práctico si hay muchos equipos pero pocas conexiones directas.

Ejemplo base en C++

```
vector<vector<int>> red(4);

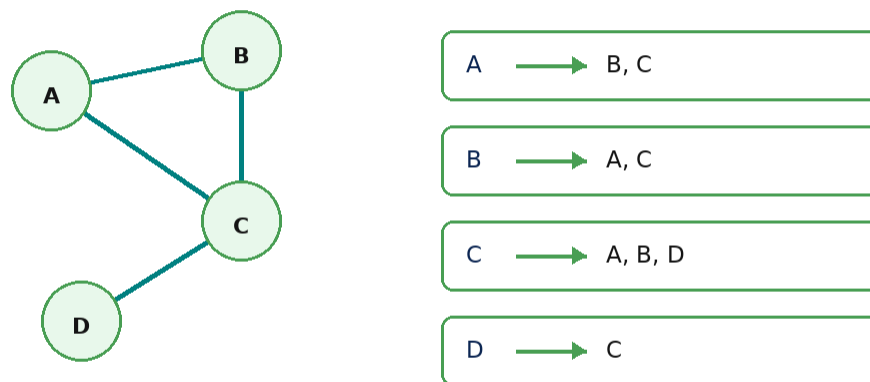
// 0=Router, 1=PC1, 2=PC2, 3=Servidor
red[0] = {1, 2, 3};
red[1] = {0};
red[2] = {0, 3};
red[3] = {0, 2};

for (int vecino : red[0]) {
    cout << "Router conectado con nodo " << vecino << endl;
}
```

Imagen de apoyo visual

c++ Lista de adyacencia

Lista de adyacencia



Solo se guardan las conexiones existentes. Es útil para grafos grandes con pocas aristas, como redes de equipos o rutas disponibles.

Imagen 9. Lista de adyacencia: cada nodo guarda sus vecinos.

11. Aplicaciones reales de estructuras no lineales

Teoría orientada a C++

Las estructuras no lineales permiten representar problemas reales de manera más natural. Los árboles sirven para jerarquías; los grafos sirven para redes y relaciones múltiples; los grafos dirigidos acíclicos sirven para dependencias, porque indican qué proceso debe ocurrir antes que otro.

Aplicación PAD-Bienestar: En PAD-Bienestar, los árboles pueden organizar sedes, áreas y equipos; los grafos pueden representar la red de conexión; y los grafos dirigidos pueden representar dependencias del proceso de

atención o validación.

Ejemplo base en C++

```
// Dependencias del flujo de atención
// A -> B significa: A debe ocurrir antes que B.
vector<pair<string, string>> flujo = {
    {"Validar usuario", "Asignar turno"},
    {"Asignar turno", "Asignar equipo"},
    {"Asignar equipo", "Iniciar sesion"},
    {"Iniciar sesion", "Cerrar sesion"}
};
```

Imagen de apoyo visual

c++ Aplicaciones reales

Aplicaciones reales en PAD-Bienestar

Jerarquías

Árbol

Módulos del sistema: PAD -> sede -> equipo -> sesión

Redes

Grafo

Equipos, routers y servidores conectados entre sí

Dependencias Grafo dirigido

Validaciones: registrar usuario -> asignar turno -> abrir sesión

Idea: elegir la estructura depende de la forma real de las relaciones entre los datos.

Imagen 10. Aplicaciones reales de árboles, grafos y dependencias en PAD-Bienestar.

12. Cierre didáctico

El Módulo 3 permite comprender que no todos los problemas se resuelven con arreglos, listas, pilas o colas. Cuando la información tiene niveles, dependencias o múltiples conexiones, conviene analizar estructuras no lineales como árboles y grafos.

En C++, estas estructuras pueden construirse mediante struct, apuntadores, funciones recursivas, arreglos, matrices y vectores. Los árboles ayudan a organizar jerarquías; los árboles binarios de búsqueda facilitan búsquedas ordenadas; los recorridos permiten visitar todos los nodos; y los grafos permiten representar redes, rutas y dependencias.

Conclusión clave: Para el incidente crítico PAD-Bienestar, las estructuras no lineales amplían el criterio de

diseño: permiten modelar jerarquías del sistema, conexiones de red y dependencias del flujo de atención. Aunque la solución principal pueda usar estructuras lineales, comprender árboles y grafos ayuda a elegir mejores modelos cuando el problema se vuelve más complejo.

Fuentes de apoyo

Universidad Nacional Rosario Castellanos. (2026). Estructura de Datos. Módulo 3: Introducción a estructuras no lineales. Contenido Nuclear.

Deitel, P. J., & Deitel, H. M. (2014). C++ Cómo programar (9.^a ed.). Pearson.

Joyanes Aguilar, L. (2017). Fundamentos de programación: algoritmos, estructuras de datos y objetos (4.^a ed.). McGraw-Hill.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2012). Introducción a los algoritmos (3.^a ed.). MIT Press.

Material de apoyo didáctico elaborado para orientar el Módulo 3 al lenguaje C++ y al incidente crítico PAD-Bienestar.